2 Obiettivi

La società SoftEngUniNA ha l'obiettivo di commercializzare diversi sistemi informatici.

Per la realizzazione del suddetto obiettivo, SoftEngUniNA richiede la progettazione, implementazione e parziale verifica dei moduli software necessari al corretto funzionamento dei sistemi informatici. Le linee di attività richieste sono descritte di seguito, mentre un elenco dettagliato degli artefatti da realizzare e consegnare per ciascuna attività è specificato in **Sezione 5**.

2.1 Analisi e specifica dei requisiti

Obiettivo di questa linea di attività è la fornitura di un documento che contenga la definizione, attraverso opportuni strumenti di formalizzazione, delle funzionalità del sistema. In particolare, si richiede la fornitura di un documento che specifichi le funzionalità del sistema attraverso notazioni UML, descrizioni testuali strutturate (es.: template di A. Cockburn), e prototipazione delle interfacce utente. Si richiede inoltre una definizione degli utenti target. Se necessario, altri modelli di dominio di analisi, possono essere impiegati per migliorare la formalizzazione del problema.

2.2 Progettazione del sistema

Obiettivo di questa linea di attività è la fornitura di un documento che descriva in maniera dettagliata la progettazione del sistema per le funzionalità richieste, attraverso modelli e notazioni UML. In particolare, i Gruppi di lavoro contraenti devono motivare e dettagliare le proprie scelte sia per quanto riguarda aspetti architetturali e tecnologici, sia per quanto riguarda la progettazione delle interfacce utente.

2.3 Implementazione del sistema

Obiettivo di questa linea di attività è la fornitura di codice sorgente e altri artefatti necessari alla compilazione ed al deployment del sistema che implementa le funzionalità richieste. L'implementazione va realizzata utilizzando un **linguaggio Object-Oriented a scelta del Contraente**, e deve soddisfare gli ulteriori requisiti dettagliati in **Sezione 3**.

2.4 Testing e valutazione dell'usabilità

Obiettivo di questa linea di attività è fornitura di un documento e di codice sorgente per la parziale verifica del sistema. In particolare, si richiede, la realizzazione di una batteria di test automatici per alcuni metodi a scelta del Contraente, sviluppata utilizzando framework della famiglia xUnit. Si richiede inoltre una valutazione dell'usabilità, realizzata sia tramite ispezioni o review sistematiche basate su checklist, che tramite il coinvolgimento di utenti reali o potenziali per condurre test di usabilità sul prodotto finito, osservando direttamente le interazioni degli utenti con il sistema e proponendo un survey di valutazione.

A. Analisi e specifica dei requisiti, con definizione di un piano di testing a livello di sistema

B. Definizione dell'architettura e progettazione del sistema

C. Implementazione del sistema

D. Definizione di test automatici di unità, e analisi dell'usabilità sul campo

3 Requisiti

DietiEstates25 è una piattaforma per la gestione di servizi immobiliari. Il sistema permette a più agenzie di pubblicare inserzioni immobiliari. Gli utenti possono quindi visualizzare le inserzioni, prenotare visite e fare offerte per acquistare/affittare un immobile. Il sistema deve consistere in un'applicazione (mobile, desktop o web-based) performante e affidabile, attraverso cui gli utenti possono fruire delle funzionalità in modo intuitivo, rapido e piacevole.

3.1 Requisiti del sistema *DietiEstates25*

Le principali funzionalità offerte da *DietiEstates25* sono indicate di seguito:

1. Ogni installazione di DietiEstates25 include un'utenza di amministrazione per il gestore dell'agenzia immobiliare, che viene creata con credenziali predefinite. Dopo aver effettuato l'accesso, è possibile modificare la password di amministrazione. L'amministratore può creare altri account di supporto all'amministrazione. I gestori di agenzie immobiliari possono a loro volta creare account per gli agenti immobiliari. Un utente interessato ai servizi immobiliari può registrarsi con un'e-mail e una password. È apprezzata la possibilità di effettuare la registrazione/login tramite credenziali di terze parti (e.g.: accedi con account Google, Facebook o GitHub). Gli utenti possono effettuare il login e accedere all'applicativo. Le credenziali devono essere salvate in modo sicuro.
2. Gli agenti immobiliari possono caricare nuovi immobili con una serie di dettagli: foto, descrizione, prezzo, dimensioni, indirizzo, numero di stanze, piano, presenza di ascensore, classe energetica, ulteriori servizi presenti (portineria, climatizzazione, ...), ecc. Gli immobili devono essere categorizzati in "vendita", "affitto". Forse in futuro sarà introdotto il supporto per affitti brevi e "case vacanze" (ma questa funzionalità al momento non è prevista). Ogni immobile inoltre è associato alla posizione geografica esatta, individuata interagendo con una mappa interattiva (tipo Google Maps).
3. Gli utenti possono eseguire una ricerca avanzata di immobili con parametri multipli: tipologia di inserzione, prezzo minimo e massimo, numero di stanze, classe energetica, posizione, ecc. In particolare, deve essere possibile effettuare filtraggi per posizione geografica almeno al livello di comune o città. È apprezzata – ma non obbligatoria – la possibilità di effettuare ricerche di immobili che si trovano in un raggio arbitrario da un certo punto specificato dall'utente e selezionato su una mappa interattiva. La ricerca deve essere efficiente e performante. Gli immobili individuati nelle ricerche vengono visualizzati su una mappa interattiva (tipo Google Maps).

8. Gli utenti possono fare un'offerta su un immobile, specificando un prezzo eventualmente inferiore a quello riportato nell'inserzione. Gli agenti immobiliari possono accettare o rifiutare l'offerta, o fare una controproposta. Deve esserci un tracking delle offerte fatte e ricevute, magari uno storico visibile sia agli agenti sia ai clienti. Gli agenti hanno la possibilità di inserire manualmente offerte ricevute al di fuori del sistema.
9. Quando viene creata un'inserzione, il sistema verifica automaticamente, tramite API di servizi esterni come Geoapify, la presenza di scuole, parchi pubblici, o fermate del trasporto pubblico nei pressi della posizione dell'immobile. Nel caso di riscontro affermativo, un apposito indicatore viene associato all'inserzione ("Vicino a scuole", "Vicino a parchi", "Vicina a trasporto pubblico").

3.2 Ulteriori requisiti

Tutte le funzionalità assegnate devono essere realizzate implementando **un sistema distribuito** che preveda due macro-componenti indipendenti:

- **Back-end:** Almeno un componente che gestisce la logica di business e i dati, esponendo interfacce di programmazione accessibili via rete (ad esempio, tramite API REST). Il back-end deve essere completamente indipendente dalla parte front-end, ovvero, deve poter funzionare ed essere eseguito in maniera autonoma, senza alcuna conoscenza o dipendenza diretta da come il front-end verrà implementato.
- **Front-end:** Un'interfaccia utente che si appoggia ai servizi offerti dal back-end esclusivamente attraverso la rete. Il front-end deve comunicare con il back-end solo tramite le API esposte, e deve poter essere sostituito o modificato senza la necessità di modifica alcuna sul back-end. Analogamente, il front-end deve poter essere eseguito separatamente dal back-end.

Il back-end è responsabile della gestione centralizzata dello stato del sistema, garantendo che tutte le informazioni e i dati persistenti siano conservati e aggiornati unicamente su di esso. È altresì auspicabile (ma non obbligatorio) che il back-end sia distribuito utilizzando tecnologie di containerizzazione come Docker e che, al momento della discussione del progetto con demo del prodotto, sia messo in opera utilizzando tecnologie allo stato dell'arte quali ad esempio servizi di public Cloud Computing come Azure o AWS, e sia quindi accessibile attraverso la rete Internet.

Per lo svolgimento delle attività di progettazione, è obbligatorio l'utilizzo di tool di CASE. Inoltre, si richiede tassativamente di astrarre il design per favorire il riutilizzo del codice e la futura implementazione di altre funzionalità, **esplicitando nella documentazione** (si veda **Sezione 5**) **le scelte intraprese per favorire tale astrazione**.

Per quanto riguarda le tecnologie da utilizzare, è data **piena libertà** di scelta al Gruppo di Lavoro contraente, **con l'unico vincolo dell'utilizzo obbligatorio di linguaggi di programmazione Object-Oriented**. Per esempio, la parte front-end può essere realizzata come applicazione desktop (e.g.: con Java + Swing/JavaFx), come applicazione web (utilizzando framework per Single Page Applications, come Angular o React), oppure come applicazione mobile (per esempio per dispositivi Android o iOS). Il Gruppo di Lavoro dovrà motivare le proprie scelte tecnologiche in fase di discussione del prodotto (vedi **Sezione 9**).

Si sottolinea che le fasi di analisi e progettazione object-oriented del sistema sono fondamentali per una valutazione positiva del Progetto. L'utilizzo di servizi **MBaaS** (come Firebase, Supabase, e simili) non sostituisce in alcun modo queste fasi. Pertanto, **non saranno accettati progetti in cui la logica applicativa e la persistenza dei dati sono gestite esclusivamente tramite servizi esterni**, poiché ciò renderebbe il sistema troppo dipendente da un singolo provider esterno.

5 Output attesi dal Committente

Per le attività sopracitate, il Committente richiede i seguenti output:

1. **Documento dei Requisiti Software**
    a. Glossario.
    b. Modellazione di tutti i casi d'uso richiesti tramite Use Case Diagrams.
    c. Individuazione e caratterizzazione del target degli utenti, tramite Personas.
    d. Descrizione dei requisiti non-funzionali e di dominio.
    e. Formalizzazione di almeno quattro **(due, se si consegna prima del 24/03/2025)** casi d'uso significativi (esclusi quelli relativi a registrazione/autenticazione utenti) a scelta dei contraenti tra quelli richiesti. Per ciascuno di questi casi d'uso, è richiesta:
        i. Descrizione testuale strutturata, realizzata utilizzando un formalismo tabellare, come quello di A. Cockburn.
        ii. Prototipazione visuale via Mock-up delle relative interfacce utente, attraverso strumenti di *rapid prototyping*.
2. **Documento di Design del sistema.**
    a. Descrizione dell'architettura proposta, con esplicita definizione dei criteri di design adottati e delle motivazioni dietro tale scelta.
    b. Descrizione e motivazione delle scelte tecnologiche adottate.
    c. Descrizione dello schema per la persistenza dati.
    d. Descrizione e motivazione delle scelte di design dell'interfaccia utente adottate.
    e. Diagramma delle classi di design.
    f. Diagrammi di sequenza di design per i casi d'uso identificati al punto **1.e**.
3. **Artefatti Software e documentazione del processo di sviluppo.**
    a. Codice Sorgente sviluppato, comprensivo di eventuale Dockerfile.
    b. File di build automatica
    c. Evidenza dell'uso di strumenti di versioning (e.g.: descrizione del sistema di versioning utilizzato, report con statistiche come numero di commit e di contributors, frequenza dei commit, etc..).
    d. Report di qualità del codice, generati da SonarQube o simili (soltanto per il back-end).
4. **Testing e valutazione dell'usabilità.**
    a. Codice xUnit per *unit testing* di quattro **(due, se si consegna prima del 24/03/2025)** metodi non banali, **che abbiano almeno due parametri**. In aggiunta al codice di test, una apposita sezione della documentazione deve descrivere le strategie adottate per la progettazione dei test dei (e.g.: classi di equivalenza individuate e coperte, criteri di copertura strutturale adottati, etc.).
    b. Descrizione delle tecniche di valutazione dell'usabilità applicate, che devono includere almeno:
        i. *Expert reviews / inspections*: definizione di una checklist per review sistematica dell'usabilità del prodotto finito, e applicazione della stessa al prodotto.
        ii. Progettazione e conduzione di un esperimento con utenti reali o potenziali per la valutazione dell'usabilità del sistema realizzato.
            1. Descrizione dei soggetti reclutati, della procedura sperimentale e delle metriche considerate.
            2. Definizione di un survey da sottoporre agli utenti dopo l'esperimento.
            3. Presentazione e discussione dei risultati dell'esperimento e del survey.

Si noti che i Punti 1, 2 e 4 vanno realizzati come capitoli distinti in un unico documento, di seguito indicato come "**documentazione**".